

Sprawozdanie

Nr laboratorium: 5

Temat: Całkowanie numeryczne (Metoda prostokątów i biblioteka GSL)

Autor: Vladyslav Humeniuk

Data: 15.11.2025

1. Co było do zrobienia

Celem laboratorium było porównanie efektywności prostych metod całkowania numerycznego z zaawansowanymi algorytmami dostępnymi w bibliotece GSL.

Zadania szczegółowe obejmowały:

- Analizę analityczną:** Obliczenie dokładnych wartości całek dla funkcji:
 - $f_1(x) = x^2 + x + 1$ na przedziale $[0, 1]$
 - $f_2(x) = \sqrt{1 - x^2}$ na przedziale $[0, 1]$
 - $f_3(x) = \frac{1}{\sqrt{x}}$ na przedziale $[0, 1]$
 - Implementację własną:** Napisanie programu obliczającego te całki metodą prostokątów (wariant punktu środkowego) i zbadanie, jak szybko błąd maleje do zadanych tolerancji ($10^{-3}, 10^{-4}, 10^{-5}, 10^{-6}$).
 - Implementację GSL:** Wykorzystanie funkcji `gsl_integration_qags` (adaptacyjna kwadratura z obsługą osobliwości) do tych samych obliczeń i porównanie liczby wymaganych podprzedziałów.
 - Zadania dodatkowe:** Obliczenie całek dla funkcji trygonometrycznych i logarytmicznych ($\sin x, \tan x, \ln(x + x^2)$) przy użyciu GSL.
-

2. Moje podejście do rozwiązania problemu

Do rozwiązania zadania wykorzystano język C.

Metoda Prostokątów (Midpoint Rule):

Zaimplementowano metodę punktu środkowego:

$$\int_a^b f(x)dx \approx h \sum_{i=0}^{n-1} f(a + (i + 0.5)h), \quad \text{gdzie } h = \frac{b - a}{n}$$

Wybór punktu środkowego ($i + 0.5$) był kluczowy dla funkcji $f_3(x) = 1/\sqrt{x}$, ponieważ pozwala uniknąć dzielenia przez zero w punkcie $x = 0$.

Algorytm szukania wymaganego N polegał na podwajaniu liczby podprzedziałów w pętli, dopóki błąd bezwzględny $|Wynik_{num} - Wynik_{dokł}|$ nie spadł poniżej progu tolerancji.

Biblioteka GSL:

Wykorzystano funkcję `gsl_integration_qags`. Jest to procedura adaptacyjna, która automatycznie zagęszcza próbkowanie w miejscach trudnych (np. duże zmiany pochodnej, osobliwości). Pozwoliło to na bezpośrednie porównanie "siłowe" metody prostokątów z metodą inteligentną.

Funkcje testowe:

- f_1 : Wielomian (gładka funkcja).
- f_2 : Ćwiartka koła (pochodna dąży do ∞ przy $x = 1$).
- f_3 : Osobliwość w punkcie $x = 0$ (wartość dąży do ∞ , ale całka jest zbieżna).

3. Implementacja (Kod programu)

Kod w języku C (`integrator.c`)

```
#include <stdio.h>
#include <math.h>
#include <gsl/gsl_integration.h>
#include <gsl/gsl_errno.h>

/*****/
/* f1: x^2 + x + 1 */
/*****/
double f1(double x, void * params)
{
    return x*x + x + 1;
}

/*****/
/* f2: sqrt(1 - x^2) */
/*****/
double f2(double x, void * params)
{
    return sqrt(1 - x*x);
}

/*****/
/* f3: 1/sqrt(x) */
/*****/
```

```

double f3(double x, void * params)
{
    if (x == 0) return 0; /* Safeguard, although midpoint method won't hit 0 */
    return 1.0 / sqrt(x);
}

/*****/
/* f_sin: sin(x) */
/*****/
double f_sin(double x, void * params)
{
    return sin(x);
}

/*****/
/* f_tan: tan(x) */
/*****/
/* */
double f_tan(double x, void * params)
{
    return tan(x);
}

/*****/
/* f_log: log(x + x^2) -> ln(x + x^2) */
/*****/
double f_log(double x, void * params)
{
    return log(x + x*x);
}

/*****/
/* Rectangle Method (Midpoint Rule) */
/*****/
double rectangle_integration(double (*f)(double, void*), double a, double b, int n)
{
    double h = (b - a) / n;
    double sum = 0.0;

    for (int i = 0; i < n; i++)
    {
        /* Midpoint of the interval */
        double x = a + (i + 0.5) * h;
        sum += f(x, NULL);
    }

    return sum * h;
}

/*****/
/* --- Function for cw1 --- */
/*****/

```

```

void analyze_function(const char* name, double (*f)(double, void*), double exact_val,
double a, double b)
{
    printf("\n== Function Analysis: %s on [%.2f, %.2f] ==\n", name, a, b);
    printf("Exact value: %.10f\n", exact_val);
    printf("%-10s | %-15s | %-15s | %-15s\n", "Tolerance", "N (Rect)", "N (GSL)", "GSL
Result");
    printf("-----\n");

    double tolerances[] = {1e-3, 1e-4, 1e-5, 1e-6};

    /* GSL workspace allocation */
    gsl_integration_workspace * w = gsl_integration_workspace_alloc (10000);
    gsl_function F;
    F.function = f;
    F.params = NULL;

    for (int i = 0; i < 4; i++)
    {
        double tol = tolerances[i];

        /* 1. Rectangle method - finding N */
        /* Double N in the loop until the error drops below tolerance */
        int n_rect = 10;
        double rect_res = 0;
        double rect_err = 1.0;

        /* Limit iterations to avoid infinite loop */
        while (n_rect < 100000000)
        {
            rect_res = rectangle_integration(f, a, b, n_rect);
            rect_err = fabs(rect_res - exact_val);
            if (rect_err < tol) break;
            n_rect *= 2;
        }

        /* 2. GSL Method (QAGS - adaptive with singularities) */
        double gsl_result, gsl_error;
        /* neval: Number of evaluations / subintervals */

        /* Use tol as epsabs (absolute error), set epsrel to 0 (or very small) */
        /* int gsl_integration_qags (const gsl_function * f, double a, double b, double
epsabs, double epsrel, size_t limit, gsl_integration_workspace * workspace, double *
result, double * abserr) */
        gsl_integration_qags (&F, a, b, tol, 1e-7, 10000, w, &gsl_result, &gsl_error);

        /* GSL returns the number of intervals in w->size */
        size_t gsl_intervals = w->size;

        printf("%-10.0e | %-15d | %-15zu | %.10f\n", tol, n_rect, gsl_intervals,
gsl_result);
    }
}

```

```

    gsl_integration_workspace_free (w);
}

/*****
/* --- Function for cw2 --- */
*****/

void analyze_extra(const char* name, double (*f)(double, void*), double a, double b)
{
    printf("\n=== cw2 analysis: %s on [%.2f, %.2f] ===\n", name, a, b);
    printf("%-10s | %-15s | %-15s | %-15s\n", "Tolerance", "GSL Result", "GSL Error",
    "Intervals");

    double tolerances[] = {1e-3, 1e-4, 1e-5, 1e-6};
    gsl_integration_workspace * w = gsl_integration_workspace_alloc (10000);
    gsl_function F;
    F.function = f;
    F.params = NULL;

    for (int i=0; i<4; i++)
    {
        double result, error;
        /* Turn off error handler so tan(x) doesn't crash the program immediately */
        gsl_set_error_handler_off();

        int status = gsl_integration_qags (&F, a, b, tolerances[i], 0, 10000, w,
        &result, &error);

        if (status)
        {
            printf("%-10.0e | Error: %s\n", tolerances[i], gsl_strerror(status));
        } else {
            printf("%-10.0e | %-15.8f | %-15.8e | %zu\n", tolerances[i], result, error,
            w->size);
        }
    }

    gsl_integration_workspace_free (w);
}

int main()
{
    /* cw1 */
    analyze_function("x^2 + x + 1", f1, 11.0/6.0, 0, 1);
    analyze_function("sqrt(1 - x^2)", f2, M_PI/4.0, 0, 1);
    analyze_function("1/sqrt(x)", f3, 2.0, 0, 1);

    /* cw2 */
    analyze_extra("sin(x)", f_sin, 0, M_PI);
    analyze_extra("tan(x)", f_tan, 0, M_PI/2.0);
    analyze_extra("log(x + x^2)", f_log, 1, 4);

    return 0;
}

```



4. Wyniki

Część I: Porównanie Metody Prostokątów i GSL

1. Funkcja: $x^2 + x + 1$ (Wielomian)

Wartość dokładna: $11/6 \approx 1.8333333333$

Tolerance	N (Rect)	N (GSL)	GSL Result
10^{-3}	10	1	1.8333333333
10^{-4}	40	1	1.8333333333
10^{-5}	160	1	1.8333333333
10^{-6}	320	1	1.8333333333

2. Funkcja: $\sqrt{1 - x^2}$ (Osobliwość pochodnej)

Wartość dokładna: $\pi/4 \approx 0.7853981634$

Tolerance	N (Rect)	N (GSL)	GSL Result
10^{-3}	20	3	0.7853990101
10^{-4}	160	6	0.7853982008
10^{-5}	640	6	0.7853981634
10^{-6}	2560	6	0.7853981634

3. Funkcja: $1/\sqrt{x}$ (Osobliwość funkcji w 0)

Wartość dokładna: 2.0000000000

Tolerance	N (Rect)	N (GSL)	GSL Result
10^{-3}	655 360	6	2.0000000000
10^{-4}	41 943 040	6	2.0000000000
10^{-5}	167 772 160	6	2.0000000000
10^{-6}	167 772 160	6	2.0000000000

Część II: Zadania dodatkowe (GSL)

1. Funkcja: $\sin(x)$ na $[0, \pi]$

GSL bez problemu obliczył wynik równy **2.0** przy użyciu zaledwie **1 interwału** dla każdej tolerancji.

Tolerance	GSL Result	GSL Error	Intervals
1e-03	2.00000000	2.22044605e-14	1
1e-04	2.00000000	2.22044605e-14	1
1e-05	2.00000000	2.22044605e-14	1
1e-06	2.00000000	2.22044605e-14	1

2. Funkcja: $\tan(x)$ na $[0, \pi/2]$

Wynik programu:

Error: singularity or extremely bad function behavior detected

Jest to wynik poprawny, ponieważ całka $\int_0^{\pi/2} \tan(x) dx$ jest **rozbieżna** (dąży do $+\infty$). GSL poprawnie wykrył niemożliwość scałkowania tej funkcji w zadanym przedziale.

Tolerance	GSL Result	GSL Error	Intervals
1e-03	Error: singularity or extremely bad function behavior detected		
1e-04	Error: singularity or extremely bad function behavior detected		
1e-05	Error: singularity or extremely bad function behavior detected		
1e-06	Error: singularity or extremely bad function behavior detected		

3. Funkcja: $\ln(x + x^2)$ na $[1, 4]$

Obliczona wartość: **6.20607265**. GSL potrzebował tylko 1 interwału, co świadczy o tym, że funkcja jest dobrze uwarunkowana w tym przedziale.

Tolerance	GSL Result	GSL Error	Intervals
1e-03	6.20607265	1.00014869e-12	1
1e-04	6.20607265	1.00014869e-12	1
1e-05	6.20607265	1.00014869e-12	1
1e-06	6.20607265	1.00014869e-12	1

5. Wnioski

- Efektywność GSL:** Algorytmy adaptacyjne (GSL QAGS) są drastycznie bardziej wydajne niż metody stałokrokowe (prostokątów). Dla funkcji z osobliwością $(1/\sqrt{x})$, metoda prostokątów potrzebowała **ponad 167 milionów** podprzedziałów, aby osiągnąć dokładność 10^{-5} , podczas gdy GSL osiągnął to przy użyciu zaledwie **6 podprzedziałów**.

2. **Wpływ osobliwości:** Proste metody numeryczne (prostokątów, trapezów) słabo radzą sobie z funkcjami, które mają osobliwości lub nieskończone pochodne na krańcach przedziału. Błąd maleje tam bardzo wolno. Metody adaptacyjne wykrywają te obszary i zagęszczają siatkę tylko tam, gdzie to konieczne.
 3. **Wielomiany:** Dla funkcji gładkich (wielomianów) obie metody działają poprawnie, choć GSL wciąż jest szybszy (często wystarczy mu 1 interwał dzięki wykorzystaniu kwadratur Gaussa wysokiego rzędu).
 4. **Detekcja błędów:** Biblioteka GSL posiada mechanizmy bezpieczeństwa. W przypadku całki rozbieżnej ($\tan x$), program nie zawiesił się w nieskończonej pętli ani nie zwrócił losowej dużej liczby, lecz zgłosił błąd, co jest kluczowe w profesjonalnych zastosowaniach.
-

6. Bibliografia

1. [Dokumentacja GSL: Numerical Integration](#)
2. [Ważniak MIMUW: Całkowanie numeryczne](#)
3. [Press, W.H., et al. "Numerical Recipes in C".](#)